

MusicGen for the Generation of Repeating Ambient Audios

Dennis Farmer

Department of Performing Arts Technology, University of Michigan

1 INTRODUCTION

1.1 Objectives and Motivations

I found that I really enjoy the types of ambient audios created by "Ambient Outpost", who in their words 'creates narrative-driven ambient soundscapes shaped by visual storytelling'. In my words, his ambient tracks mix synthesizer pads with sound effects to create really effective tracks to serve as background noise, evoking imagination and sparking creativity. Through this project, I aimed to gain some familiarity with using an open-source music generation model such as MusicGen [1] to aid in novel ambience generation, in an easy to use interface that allowed for customization towards user's preferences in timbre of their ambient soundtracks.

I have created a tool called Ambigen for creating remixes of uploaded ambient synthesizer tracks, using MusicGen, an open source transformer-based AI model that generates music from a text prompt paired with an audio sample to condition on.

The primary purpose of the web app is to allow users to generate a soundtrack to listen to while working on other tasks, based on an uploaded mp3 audio file and a text prompt that can be used to guide the model towards the user's preferences for ambient music.

2 METHODS

2.1 Web Dashboard

The web interface was developed using Dash, a Python library that allows for the creation of web applications using Python to define layouts, callbacks, and components, which are then converted to HTML and React in the browser. An especially useful functionality was Dash's ability to conditionally render elements when certain events occur, which I used to show the filename circled in green when uploaded, and to prevent submitting the form until both a conditioning mp3 and a text prompt are provided. The audio then is generated, and the audio element url is updated as "/assets/ambient_synthesizer.wav?t=23891" or some other ?t= value, to refresh the browser's cache and force a re-render of the audio player, which plays automatically and loops infinitely until paused.

Some processing of the input and output of MusicGen:

1. The last 10 seconds of the prompt waveform was removed, so that the model does not condition on the uploaded track fading out to zero gain, which is often the case if an uploaded track is an entire song. Usually it is best to simply crop a section of the song prior to uploading; despite MusicGen conditioning on the entirety of the input audio, there is emphasis on the audio immediately preceding where the generation continues off from.

2. The generated track was repeated using a fade in / fade out windowing approach, to create a more convincing ambient effect.

2.2 MusicGen

The core of the project involves utilizing MusicGen, an open-source music generation model developed by Meta AI. Here, I'll provide an overview of the key aspects that make up MusicGen's approach to audio generation, most notably how the audio is encoded into tokens to be used for generating sequential tokens. Some details of the implementation are relevant in later sections of this report, where I diagnose potential areas for improvement in the application of MusicGen to generating ambient audios.

2.3 EnCodec

The original representation of raw audio samples is not very usable as is since there is such a massive number of samples with little contextual information, so instead a neural codec extracts contextually relevant features from the signal and transforms it to a discrete representation in a much smaller latent space. Meta AI's EnCodec [2] serves as a neural codec to re-encode the input audio originally at 32 kHz mono into a continuous latent sequence at a much lower 50 Hz. EnCodec then uses Residual Vector Quantization (RVQ, also used by the SoundStream neural codec covered in class) to map each continuous latent vector to $K=4$ discrete indices, one from each of K collections each containing discrete codes, with the collections being called "codebooks". This results in a matrix Q , with K columns being parallel discrete token sequences that represent the input audio sample, with each sequence being of length $d \cdot 50$ where d is the duration of the audio in seconds (50 values for each 1 second of audio, e.g. 50 Hz).

Each codebook serves as a lookup table with M learned vectors representing audio snippets. The values of the token sequences in the quantization Q are then indices pointing at one of the M vectors in the given codebook, with values 1, ..., M .

With music being a continuous process that is being modeled by discrete tokens, there is a lot of complexity that goes into the process of quantizing continuous audio well, while preserving fidelity. With generating audio especially, small errors introduced in quantization can lead to degeneration of the output, since sequential portions of the audio are being generated auto-regressively and will depend on larger and larger errors. If we want to utilize the self-attention and cross-attention afforded by transformers however, we won't be able to get around having to represent the audio in some sort of discrete form to form tokens. Residual Vector Quantization allows the model to both approximate the underlying continuous latent space of audio, and work with a discrete representation that lends itself to tokenization.

2.4 Residual Vector Quantization

The basic idea of RVQ [3] is that it reduces the impact of accumulating errors from quantization. Typical vector quantization can be thought of as utilizing one large codebook, where the codebook maps vector components to the centroid of whichever partition of the space the components lie in. Residual vector quantization instead uses parallel streams of codebooks which are in descending order of importance, with the first codebook being the most important and performing the same role of approximating the input vector. The second codebook is essentially a codebook within the first codebook, that models the residual between the original vector and the first codebook's quantization. This serves as error correction, so that quantization errors do not accumulate and compound over time. Later codebooks repeat this process to handle the residual error from the preceding codebook. Utilizing several parallel dependent streams is primarily useful to capture detail in the audio hierarchically, with the first codebook being coarse detail and later codebooks capturing finer details.

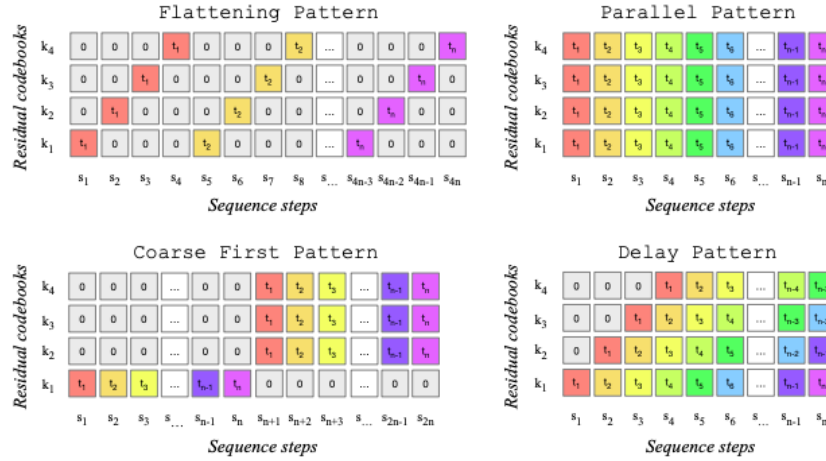


Figure 1. Each codebook stream after k_1 models the error of the previous codebook. k_2, k_3 , and k_4 can be computed in parallel as seen in the delay pattern, saving considerable computation time over the flattening pattern.

The paper goes into detail about the tradeoffs and advantages of various codebook interleaving patterns, the most effective of which was their "delay" interleaving pattern, inspired by a similar paper that applied multi-streams to speech tokens. In the delay pattern, the first codebook is computed sequentially, then the remaining codebooks are computed in parallel, each with an offset of one time step to allow computing the necessary residual of the previous notebook, which saves on computation time despite having much more information than using only a single codebook stream might. Their parallel interleaving pattern also extends similarly to generating stereo audio at no extra computational cost by delaying the generation of left and right channels by a single time step.

2.5 Transformer and Decoder

After tokenization, the input is fed into a transformer [4] with architecture described in the MusicGen paper. Most notable I think is how MusicGen handles multi-modality between audio and text. MusicGen's transformers contain cross-attention blocks to utilize the conditioned text after it is converted to semantic embeddings using Contrastive Language-Audio Pretraining (CLAP). The melody conditioning is implemented as part of the audio input, after some processing to reduce the chromagram to a simplified version containing only the peak frequency for each time step.

Decoding of the audio tokens using EnCodec consists of looking up indices in their corresponding codebook, and then combining the K layers via summation into a latent sequence at the 50 Hz rate from the encoding stage, and then reconstructing with a convolutional decoder that progressively up-samples back to the original 32 kHz sample rate. The training of the EnCodec is done using a GAN-style discriminator.

3 DISCUSSION

3.1 Key Findings and Interpretations

MusicGen provides three pre-trained models: a small 300M parameter model, a medium 1.5B parameter model, and a large 3.3B parameter model. Generally, I found that the medium and large models performed identically in terms of output quality, where the small model had a noticeable amount of noise and artifacts, which are easy for humans to notice in music.

Subjectively, the generated audios varied widely in quality. Prompt quality and input waveform were the two aspects that I varied. Despite the paper noting that the large model followed the prompt more closely, I found that the medium and large models performed mostly identically on detailed textual prompts (with large having higher sound quality). Longer prompts with more keywords created slightly richer responses (for example: "ambient synthesizer, with warm pads, slow chord changes, counterpoint, constant motion, sustain, brian eno"), whereas a cropped version like "ambient synthesizer, with warm pads" sounds more generic. More exploration could be done on writing quality text prompts, especially since my prompts all consisted of collections of keywords, however fine tuning the model itself may be more beneficial.

The manually written fade in and fade out of repetitions of the generated audio served as an effective substitute for the model's ability to generate long sustained audios. The resulting effect ends up being similar to a ocean wave on the beach or a gentle morning ringtone, with the audio not becoming too much of an ear sore over a prolonged listening session.

3.2 Future Directions

One aspect of the project that would be great to explore would be fine tuning, since the MusicGen repository has a workflow established for doing so. The reason fine tuning would be applicable is that, rather ironically for this project, MusicGen seems to not handle conditioning on ambient music well. If the input waveform does not have a distinctive rhythm with repetition, and consists mainly of pad-y sustained notes, the resulting output is usually a single note continuation followed by silence or static.

I imagine this would have to do with the distribution of the licensed music training data, which likely consists of lots of songs with repeated rhythms and envelopes with short decays, rather than long sustained notes with evolving timbre. The codebooks then as a result have learned vectors corresponding to quick snippets, and not long stationary segments. Fine tuning the EnCodec would be a worthwhile extension to this project, to experiment with learning the ability to create more subtle changes over time and have the generated sound stay sustained.

There are optimizations that could be done to improve the user experience of the application, one of which would be to provide built-in presets that use a combination of different prompts, different conditioned audios, and different strategies of windowing generated copies to guide audio generation towards user preferences with less trials required to land on an enjoyable listening track.

3.3 Conclusion

It is surprising that a model as capable as MusicGen is able to be used easily and freely. The continuations of what might be considered typical music are a very convincing timbral copy of the conditioned audio for the first couple of seconds, and serve well as background music for ambience when repeated with overlapping windowed copies.

REFERENCES

- ^[1] Copet, J., Kreuk, F., Gat, I., Remez, T., Kant, D., Synnaeve, G., Adi, Y., and Défossez, A. (2023). Simple and controllable music generation.
- ^[2] Défossez, A., Copet, J., Synnaeve, G., and Adi, Y. (2022). High fidelity neural audio compression.
- ^[3] Hawley, S. H. (2023). Residual vector quantization - an exploration of the heart of neural audio codecs.
- ^[4] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need.